

VILNIUS GEDIMINAS TECHNICAL UNIVERSITY

Iris Valverde Blanco - 20253336

Erasmus

HOMEWORK II REPORT

MICROCONTROLLERS SYSTEMS [ELESB16612]

Supervisor: Mindaugas Jankauskas

Vilnius, 2026

TABLE OF CONTENT

Task description....3
Schematic....3
Program code....4
Explanation of work....11
Summary....12

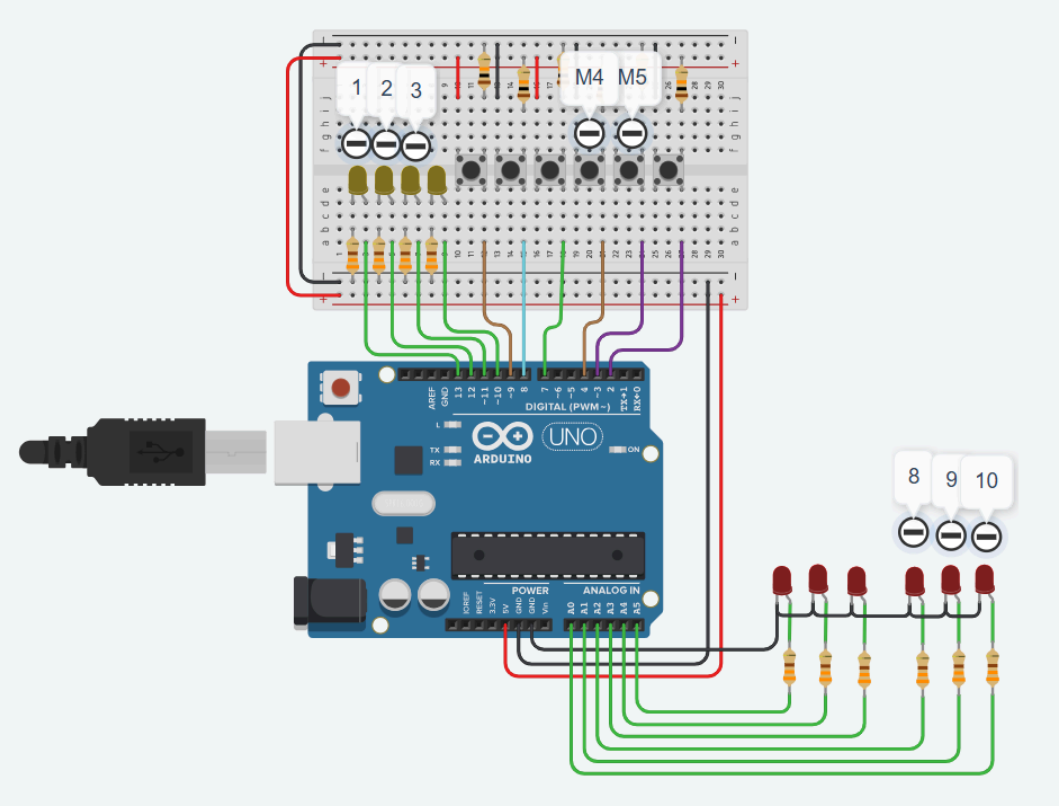
Task description

This task consists of the programming of an Arduino Uno microcontroller using plain C (AVR register programming) without using the Arduino libraries. The homework consists of two parts: the mandatory task (LEDs control using buttons), and the additional part (LCD interactive menu).

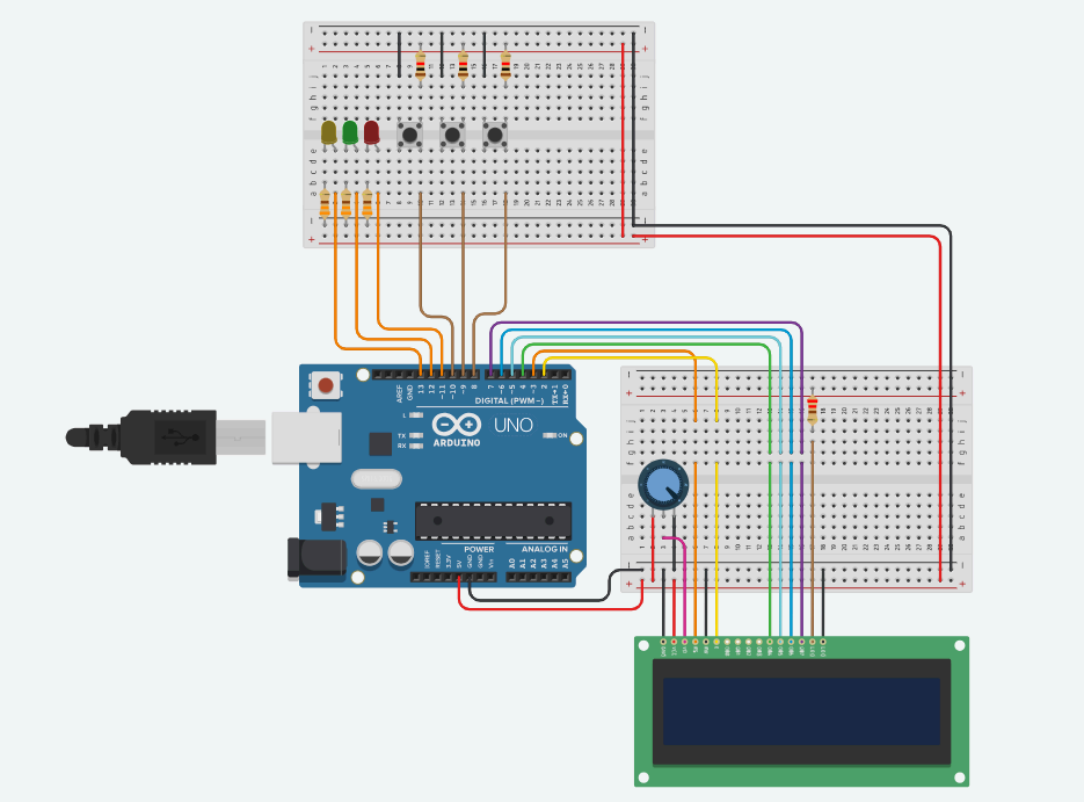
- Mandatory part: it consists of the implementation of a periodic binary sequence (001, 100, 001, 100, 011, 110, 011, 110, …) in the LEDs 8-10 changing when the button M5 is pressed, and a running line in the LEDs 1-3 that stops/restars when the button M4 is pressed.
- Additional part: it consists of creating a menu on the 16x2 LCD screen to show/control the state (ON/OFF) of three LEDs. For this, there are three buttons: M1, and M2 used for navigation (M1 selects LED to the left and M2 Selects LED to the right), and M3 toggles the current LED being selected. The screen shows on the first row the text: "VilniusTech Menu", and on the second row it shows the current LED and its state (e.g. "LED Green: ON").

Schematic

- Mandatory part:



- Additional part:



Program code

- Mandatory part:

```
C/C++
#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>

// ===== TASK 1 (sequence) =====
uint8_t seq[] = {0b001, 0b100, 0b001, 0b100, 0b011, 0b110, 0b011, 0b110};
uint8_t seq_idx = 0;

// ===== TASK 2 (running line) =====
uint8_t run_line[] = {(1 << PB5), (1 << PB4), (1 << PB3)};
uint8_t run_index = 0;
uint8_t is_running = 1;

// ===== BUTTON STATES (for edge detection) =====
uint8_t prev_M5 = 1;
```

```
uint8_t prev_M4 = 1;

// ===== TIMER =====
uint16_t tick = 0;   // increments every 10ms

void setup_registers() {
  // LED8-LED10 as outputs (PC2, PC1, PC0)
  DDRC |= 0b00000111;
  // M5 button as input (PD3)
  DDRD &= ~(1 << PD3);

  // LED1-LED3 as outputs (PB5, PB4, PB3)
  DDRB |= 0b00111000;
  // M4 button as input(PD4)
  DDRD &= ~(1 << PD4);

  // Enable pull-ups (Buttons are LOW when pressed)
  PORTD |= (1 << PD3) | (1 << PD4);
}

int main(void) {
  setup_registers();

  while (1) {
    // small fixed loop delay
    _delay_ms(10);
    tick++;

    // read the buttons
    uint8_t curr_M5 = (PIND & (1 << PD3));
    uint8_t curr_M4 = (PIND & (1 << PD4));

    // TASK 1: check if M5 is pressed (pin is low)
    if (prev_M5 && !curr_M5) {  // falling edge (pressed)
      PORTC = (PORTC & ~0x07) | seq[seq_idx];
      seq_idx = (seq_idx + 1) % 8;
    }
```

```
    // TASK 2: check if M4 is pressed (pin is low)
    if (prev_M4 && !curr_M4) {
      is_running = !is_running;
    }

    // update previous states
    prev_M5 = curr_M5;
    prev_M4 = curr_M4;

    // TASK 2: running line (every 500 ms)
    if (tick >= 50) {   // 50 * 10ms = 500ms
      tick = 0;
      if (is_running) {
        // Clear LEDs
        PORTB &= ~((1 << PB5) | (1 << PB4) | (1 << PB3));

        // Set current LED
        PORTB |= run_line[run_index];

        // Move to next
        run_index = (run_index + 1) % 3;
      }
    }
  }

    return 0;
}
```

- Additional part:

```
C/C++
/*
4-Bit 16x2 LCD
*/

//#define F_CPU 16000000UL   // Not necessary in simulator
```

```
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

// Your global variables here
int current_led = 0;                    // 0: Yellow, 1: Green, 2: Red
uint8_t states[3] = {0, 0, 0};    // 0: OFF, 1: ON
char* colors[] = {"Yellow:  ", "Green:   ", "Red:     "};

// Defination of pot and pin variables
#define LCD_Port         PORTD    // Define LCD Port (PORTA, PORTB, PORTC,
PORTD)
#define LCD_DPin         DDRD     // Define 4-Bit Pins (PD4-PD7 at PORT D)
#define LCD_DATA0_PIN    PD4    // < pin for 4bit data bit 0
#define LCD_DATA1_PIN    PD5    // < pin for 4bit data bit 1
#define LCD_DATA2_PIN    PD6    // < pin for 4bit data bit 2
#define LCD_DATA3_PIN    PD7    // < pin for 4bit data bit 3
#define RSPIN                     PD3   // RS Pin
#define ENPIN            PD2      // E Pin
int runtime;                //Timer for LCD

// LCD initialization function with the minimum commands list
void LCD_Init (void)
{
  // Data and control pins as outputs
  LCD_DPin |=
(1<<LCD_DATA0_PIN)|(1<<LCD_DATA1_PIN)|(1<<LCD_DATA2_PIN)|(1<<LCD_DATA3_PIN)|(1<
<RSPIN)|(1<<ENPIN);        //Control LCD Pins (D4-D7)
  _delay_ms(16);           //Wait more when 15 ms after start

  LCD_Action(0x02);     // Returns the cursor to the home position (Address 0).
Returns display to its original state if it was shifted.
  LCD_Action(0x28);       // Data sent or received in 4 bit lengths (DB7-DB4)
  LCD_Action(0x0C);       // Display on, cursor off
  LCD_Action(0x06);       // Increment cursor (shift cursor to right)
  LCD_Action(0x01);       // Clean LCD
  _delay_ms(5);
```

```
}

// Function to send commands and data to the LCD
void LCD_Action(unsigned char cmnd)
{
  LCD_Port = (LCD_Port & 0x0F) | (cmnd & 0xF0);
  LCD_Port &= ~ (1<<RSPIN);
  LCD_Port |= (1<<ENPIN);
  _delay_us(1);
  LCD_Port &= ~ (1<<ENPIN);
  _delay_us(200);
  LCD_Port = (LCD_Port & 0x0F) | (cmnd << 4);
  LCD_Port |= (1<<ENPIN);
  _delay_us(1);
  LCD_Port &= ~ (1<<ENPIN);
  _delay_ms(2);
}

// Clear whole LCD
void LCD_Clear()
{
  LCD_Action (0x01);       //Clear LCD
  _delay_ms(2);                  //Wait to clean LCD
  LCD_Action (0x80);       //Move to Position Line 1, Position 1
}

// Prints string of chars
void LCD_Print (char *str)
{
  int i;
  for(i=0; str[i]!=0; i++)
  {
    LCD_Port = (LCD_Port & 0x0F) | (str[i] & 0xF0);
    LCD_Port |= (1<<RSPIN);
    LCD_Port|= (1<<ENPIN);
    _delay_us(1);
    LCD_Port &= ~ (1<<ENPIN);
```

```
    _delay_us(200);
    LCD_Port = (LCD_Port & 0x0F) | (str[i] << 4);
    LCD_Port |= (1<<ENPIN);
    _delay_us(1);
    LCD_Port &= ~ (1<<ENPIN);
    _delay_ms(2);
  }
}

// Prints string of chars to the specific location
// row - the row 0 or 1;
// pos - the column from 0 till 16
void LCD_Printpos (char row, char pos, char *str)
{
  if (row == 0 && pos<16)
    LCD_Action((pos & 0x0F)|0x80);
  else if (row == 1 && pos<16)
    LCD_Action((pos & 0x0F)|0xC0);
  LCD_Print(str);
}

void update_leds() {
  if(states[0]) PORTB |= (1 << PB5); else PORTB &= ~(1 << PB5);
  if(states[1]) PORTB |= (1 << PB4); else PORTB &= ~(1 << PB4);
  if(states[2]) PORTB |= (1 << PB3); else PORTB &= ~(1 << PB3);
}

void refresh_text() {
  LCD_Printpos(0, 0, "VilniusTech Menu");
  LCD_Printpos(1, 0, "LED ");
  LCD_Print(colors[current_led]);

  if(states[current_led]) {
    LCD_Print("ON ");
  } else {
    LCD_Print("OFF");
  }
```

```
}

void setup() {
  // Buttons M1 (PB2), M2 (PB1), M3 (PB0)
  DDRB &= ~((1 << PB2) | (1 << PB1) | (1 << PB0));
  PORTB |= (1 << PB2) | (1 << PB1) | (1 << PB0);

  // LEDs Yelloww (PB5), Green (PB4), Red (PB3)
  DDRB |= (1 << PB5) | (1 << PB4) | (1 << PB3);
}

int main()
{
  LCD_Init();

  setup();

  refresh_text();

  while(1) {
    // M1 button: select previous LED
    if (!(PINB & (1 << PB2))) {
      _delay_ms(50);
      current_led = (current_led + 2) % 3;
      refresh_text();
      while (!(PINB & (1 << PB2)));
      _delay_ms(50);
    }

    // M2 button: select next LED
    if (!(PINB & (1 << PB1))) {
      _delay_ms(50);
      current_led = (current_led + 1) % 3;
      refresh_text();
      while (!(PINB & (1 << PB1)));
      _delay_ms(50);
    }
```

```
    // M3 button: change status of current LED
    if (!(PINB & (1 << PB0))) {
      _delay_ms(50);
      states[current_led] = !states[current_led];
      update_leds();
      refresh_text();
      while (!(PINB & (1 << PB1)));
      _delay_ms(50);
    }
  }
  return 0;
}
```

Explanation of work

The project relies on "bare-metal" programming, where every hardware action is controlled by writing specific bits to memory-mapped registers.

I/O Management via Bitwise Logic

Instead of high-level functions, we utilize bitwise operators to interact with the AVR registers:

- Data Direction (DDRx): bit is set to 1 for outputs (LEDs) and bit is set to 0 for inputs (buttons).
- Internal Pull-up Resistors: by writing 1 to the PORT register of an input pin, the internal pull-up resistor is enabled. This ensures that the pin reads HIGH by default and LOW only when the button is pressed, preventing electrical noise from causing “ghost” presses.
- Port Masking: To update specific LEDs without affecting other pins on the same port (like those used for the LCD), masking is used: PORTB &= ~MASK to clear bits and PORTB |= VALUE to set them.

Non-Blocking Multitasking and Timing

To ensure the system remains responsive, I avoided long _delay_ms() calls, since they “freeze” the CPU.

- The Tick system: the main loop runs every 10ms. A counter tracks these intervals.

- Animation timing: the running line only advances when the tick reaches 50 (500ms), but the buttons are checked every 10ms. This allows that the CPU is not blocked and buttons are checked continuously while the running line appears to move slowly.

Logical Sequencing via Modulo and Bit-Shifting

To manage the multiple LED behaviours efficiently, I used mathematical and bitwise logic:

- Modulo Operator (%): it is used to handle the periodic sequences. By calculating (index + 1) % size, the program moves to the next index and returns to the beginning without using complex nested if statements.
- Bit-Shifting: to create the "running line" for LED1-LED3, the code shifts a single active bit across the register. This physically targets different pins in succession, providing a more efficient way to move light than addressing each pin individually.

LCD Communication Protocol (4-bit Mode)

Since the LCD uses a 4-bit interface to save pins, communication is handled in two stages:

1. The High Nibble (bits 7-4) is sent first, followed by a pulse on the Enable (EN) pin.
2. The Low Nibble (bits 3-0) is sent second, followed by another pulse.
3. The Register Select (RS) pin is toggled to distinguish between a command (like clearing the screen) and data (the characters to be printed).

Summary

This homework demonstrates the integration of hardware and software at a low level. By not using the Arduino abstraction layer, the resulting code is highly efficient and provides precise control over the ATmega328P architecture. The final system integrates a variety of embedded interaction methods (periodic sequences, togglable animations, and a multi-level interactive menu) into a single responsive environment.

Key Outcomes:

- The use of edge-detection logic for buttons prevents multiple triggers from a single press, ensuring a stable user experience.
- The implementation of a non-blocking timer allows the microcontroller to manage button inputs, a background animation, and a dynamic LCD menu simultaneously.
- Mastering register-level programming provides a fundamental understanding of embedded systems.